

Salahuddin Saiet

Software Developer

salahuddinsaiet.10@gmail.com · 061-403-9272 · Cape Town
github.com/Saiby100 · salahuddin-saiet.vercel.app · linkedin.com/in/salahuddin-saiet-2a7190241

Summary

Software developer with experience building internal business tooling and streamlining complex business processes. I also lead project work end to end, from planning and scoping with clients to partitioning features for my team to work as efficiently as possible.

I am experienced in agent orchestration throughout the development lifecycle in a production environment, from designing and architecting new features to implementation, reviewing and testing. My favourite tools include a highly customised tmux configuration, Neovim for coding, and Obsidian for research and notes.

Skills

Languages: Java, JavaScript, TypeScript, Python, SQL, Kotlin, C

Tools: React, Node, Next.js, FastAPI, Lowdefy, MongoDB, Docker, Git, AWS Lambda, AWS API Gateway, Claude Code, tmux, Neovim, JIRA

Practices: Agile, code review, CI/CD, database design and query optimisation, object-oriented design, testing, AI agent orchestration

Experience

Software Developer

March 2024 – Present

Lowdefy — low-code platform for internal business tools, built on React and Node

- Planning, building and maintaining full-stack web applications and internal business tooling for a range of clients, using JavaScript, React and Node. This includes the WhatsApp tooling that around 30 technicians use daily for on-site work, which runs on an AWS Lambda function behind API Gateway.
- Maintaining and improving the Lowdefy framework itself, with 19 merged PRs to the open-source repo. Work includes adding new features to the transpiler and refactoring how blocks are structured.
- Architecting scalable, maintainable solutions, including database design and query optimisation in MongoDB. Rebuilt the Kanban board so it handles 1000+ cards, up from stalling at a few hundred.
- Leading developers through the full lifecycle of new features, including allocating work, code review and QA. As tech lead on Workflows, designed the schema behind it, which cut build time per workflow from 1–2 weeks to 2–3 days.
- Providing technical insight in client-facing meetings, evaluating complexity and feasibility and setting timelines.

Computer Science Teaching Assistant

July – October 2023

Stellenbosch University

- Ran weekly practical sessions helping students complete Python problems; invigilated exams and marked project submissions and demos.

Education

BSc Mathematical Science (Computer Science)

2021 – 2023

Stellenbosch University

- Java was my first language: programming fundamentals, object-oriented programming, and data structures and algorithms.
- Live messaging app with voice calls over a local area network, built in Java as a team: text messages over TCP, voice over UDP sockets.
- Android app in Kotlin that matches a user-captured aerial map image to a real location, lines it up and overlays it on the Google Maps view, with straight-line distances and the user's live position projected onto the overlay.

- Othello game in C with a computer opponent that explores possible moves in parallel across threads, coordinating them so no two search the same line of play.
- Compiler in C for a custom language (ALAN) targeting the JVM, and networking basics: network types, DNS name resolution, and how packets move between devices.

Projects

AI Reader, document parsing service (*Python, FastAPI, Docling, RapidOCR, Docker*)

- Built a FastAPI service that parses uploaded documents, with bearer-token auth, file validation and models loaded lazily on startup.
- PDFs with a broken text layer were not parsed correctly. Recovered the lost ligatures by identifying the affected pages, re-reading them with OCR to build a mapping of each unique broken glyph, then applying that mapping across the document.
- Kept the pipeline cheap to run: OCR and formula/code enrichment default off, since the enrichment pass runs a vision-language model per block. Around 350 lines of tests over the parser. Runs as a two-service Docker Compose stack alongside the Next.js front end.

Reminder, prayer times app for South Africa (*TypeScript, React Native, Expo, Supabase, PostgreSQL*)

- Shipped an Android app that shows daily prayer times by area, with date navigation, a Hijri calendar, a qibla compass and light/dark theming.
- Backed by a Supabase Postgres database. Fetches a month of times in one query and caches it on the device, so the app stays responsive and keeps working offline.
- Schedules reminder notifications ahead of each prayer at a configurable lead time, kept current by a daily background task so the schedule survives without the app being opened.
- Ships releases through GitHub, with the app checking for and prompting to install newer versions itself.

Lowdefy LSP, VS Code language server (*TypeScript, Language Server Protocol, AJV*)

- Writing Lowdefy YAML by hand means keeping every block's schema in your head. Built a language server that reads those schemas and offers context-aware completions and inline validation as you type.
- Resolves the cursor's position in the YAML AST to an exact block path, then ranks suggestions from the matching block schema, so completions are scoped to what is actually valid at that point in the file.
- Caches a parsed AST per open document and revalidates on change, keeping completion and diagnostics responsive on large configuration files.